

# AI agent orchestration patterns

As architects and developers design their workload to take full advantage of language model capabilities, AI agent systems become increasingly complex. These systems often exceed the abilities of a single agent that has access to many tools and knowledge sources. Instead, these systems use multi-agent orchestrations to handle complex, collaborative tasks reliably. This guide covers fundamental orchestration patterns for multi-agent architectures and helps you choose the approach that fits your specific requirements.

## Overview

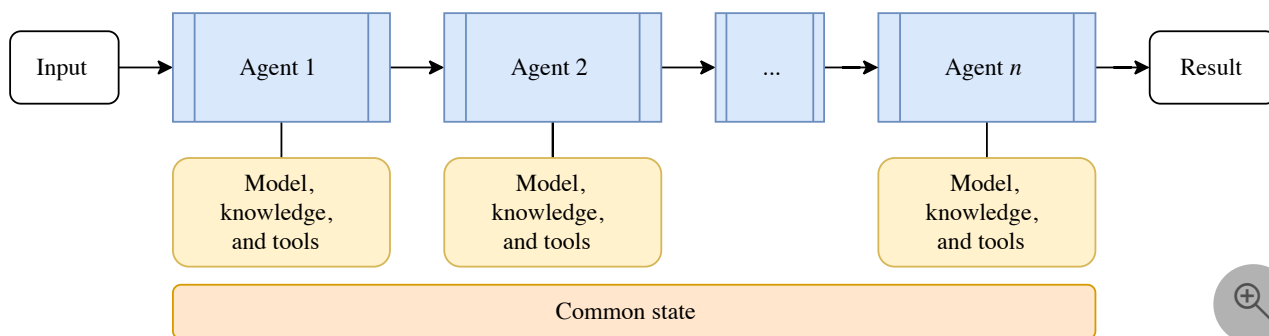
When you use multiple AI agents, you can break down complex problems into specialized units of work or knowledge. You assign each task to dedicated AI agents that have specific capabilities. These approaches mirror strategies found in human teamwork. Using multiple agents provides several advantages compared to monolithic single-agent solutions.

- **Specialization:** Individual agents can focus on a specific domain or capability, which reduces code and prompt complexity.
- **Scalability:** Agents can be added or modified without redesigning the entire system.
- **Maintainability:** Testing and debugging can be focused on individual agents, which reduces the complexity of these tasks.
- **Optimization:** Each agent can use distinct models, task-solving approaches, knowledge, tools, and compute to achieve its outcomes.

The patterns in this guide show proven approaches for orchestrating multiple agents to work together and accomplish an outcome. Each pattern is optimized for different types of coordination requirements. These AI agent orchestration patterns complement and extend traditional [cloud design patterns](#) by addressing the unique challenges of coordinating autonomous components in AI-driven workload capabilities.

## Sequential orchestration

The sequential orchestration pattern chains AI agents in a predefined, linear order. Each agent processes the output from the previous agent in the sequence, which creates a pipeline of specialized transformations.



The sequential orchestration pattern solves problems that require step-by-step processing, where each stage builds on the previous stage. It suits workflows that have clear dependencies and improve output quality through progressive refinement. This pattern resembles the [Pipes and Filters](#) cloud design pattern, but it uses AI agents instead of custom-coded processing components. The choice of which agent gets invoked next is deterministically defined as part of the workflow and isn't a choice given to agents in the process.

## When to use sequential orchestration

Consider the sequential orchestration pattern in the following scenarios:

- Multistage processes that have clear linear dependencies and predictable workflow progression
- Data transformation pipelines, where each stage adds specific value that the next stage depends on
- Workflow stages that can't be parallelized
- Progressive refinement requirements, such as *draft*, *review*, *polish* workflows
- Systems where you understand the availability and performance characteristics of every AI agent in the pipeline, and where failures or delays in one AI agent's processing are tolerable for the overall task to be accomplished

## When to avoid sequential orchestration

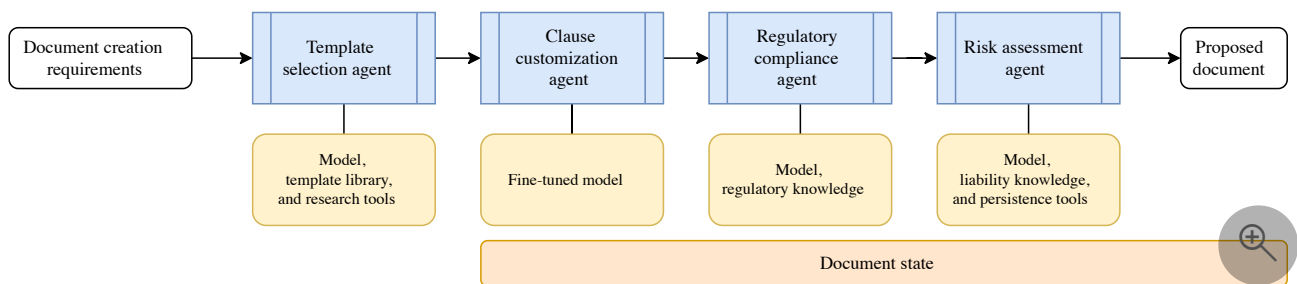
Avoid this pattern in the following scenarios:

- Stages are [embarrassingly parallel](#) [↗](#). You can parallelize them without compromising quality or creating shared state contention.
- Processes that include only a few stages that a single AI agent can accomplish effectively.

- Early stages might fail or produce low-quality output, and there's no reasonable way to prevent later steps from processing by using accumulated error output.
- AI agents need to collaborate rather than hand off work.
- The workflow requires backtracking or iteration.
- You need dynamic routing based on intermediate results.

## Sequential orchestration example

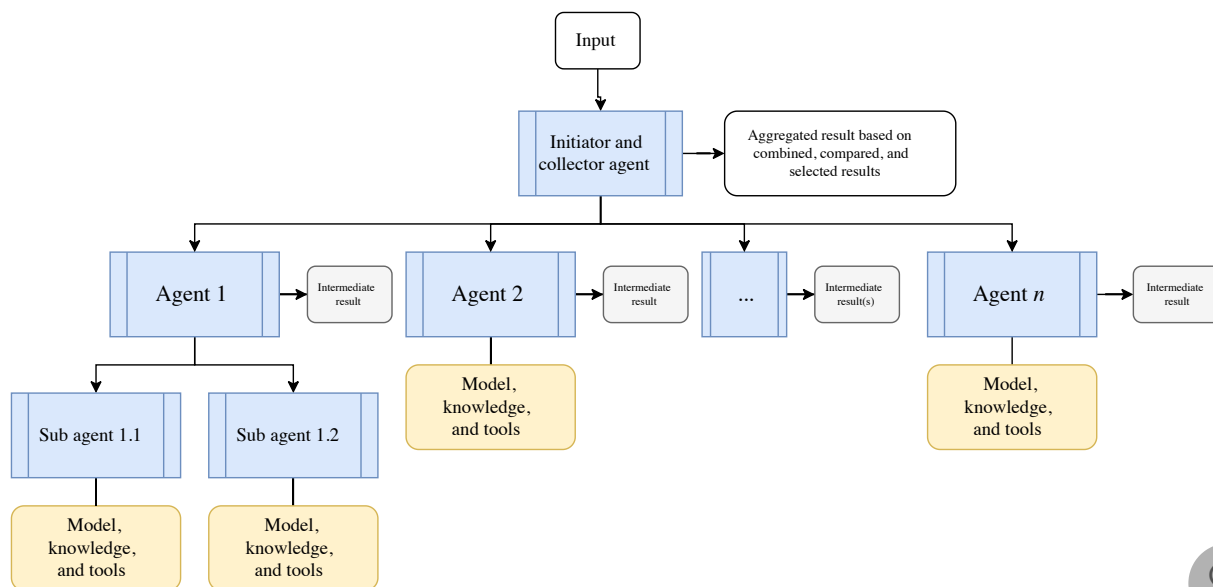
A law firm's document management software uses sequential agents for contract generation. The intelligent application processes requests through a pipeline of four specialized agents. The sequential and predefined pipeline steps ensure that each agent works with the complete output from the previous stage.



1. The *template selection agent* receives client specifications, like contract type, jurisdiction, and parties involved, and selects the appropriate base template from the firm's library.
2. The *clause customization agent* takes the selected template and modifies standard clauses based on negotiated business terms, including payment schedules and liability limitations.
3. The *regulatory compliance agent* reviews the customized contract against applicable laws and industry-specific regulations.
4. The *risk assessment agent* performs comprehensive analysis of the complete contract. It evaluates liability exposure and dispute resolution mechanisms while providing risk ratings and protective language recommendations.

## Concurrent orchestration

The concurrent orchestration pattern runs multiple AI agents simultaneously on the same task. This approach allows each agent to provide independent analysis or processing from its unique perspective or specialization.



This pattern addresses scenarios where you need diverse insights or approaches to the same problem. Instead of sequential processing, all agents work in parallel, which reduces overall run time and provides comprehensive coverage of the problem space. This orchestration pattern resembles the Fan-out/Fan-in cloud design pattern. The results from each agent are often aggregated to return a final result, but that's not required. Each agent can independently produce its own results within the workload, such as invoking tools to accomplish tasks or updating different data stores in parallel.

Agents operate independently and don't hand off results to each other. An agent might invoke extra AI agents by using its own orchestration approach as part of its independent processing. The available agents must know which agents are available for processing. This pattern supports both deterministic calls to all registered agents and dynamic selection of which agents to invoke based on the task requirements.

## When to use concurrent orchestration

Consider the concurrent orchestration pattern in the following scenarios:

- Tasks that you can run in parallel, either by using a fixed set of agents or by dynamically choosing AI agents based on specific task requirements.
- Tasks that benefit from multiple independent perspectives or different specializations, such as technical, business, and creative approaches, that can all contribute to the same problem. This collaboration typically occurs in scenarios that feature the following multi-agent decision-making techniques:
  - Brainstorming

- Ensemble reasoning
- Quorum and voting-based decisions
- Time-sensitive scenarios where parallel processing reduces latency.

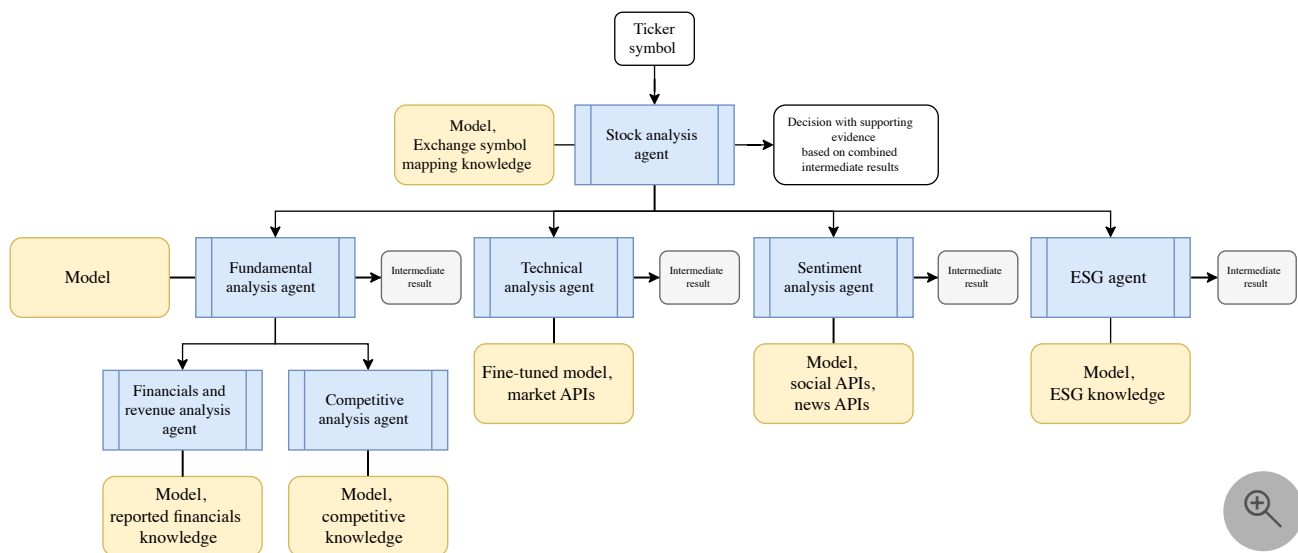
## When to avoid concurrent orchestration

Avoid this orchestration pattern in the following scenarios:

- Agents need to build on each other's work or require cumulative context in a specific sequence.
- The task requires a specific order of operations or deterministic, reproducible results from running in a defined sequence.
- Resource constraints, such as model quota, make parallel processing inefficient or impossible.
- Agents can't reliably coordinate changes to shared state or external systems while running simultaneously.
- There's no clear conflict resolution strategy to handle contradictory or conflicting results from each agent.
- Result aggregation logic is too complex or lowers the quality of the results.

## Concurrent orchestration example

A financial services firm built an intelligent application that uses concurrent agents that specialize in different types of analysis to evaluate the same stock simultaneously. Each agent contributes insights from its specialized perspective, which provides diverse, time-sensitive input for rapid investment decisions.



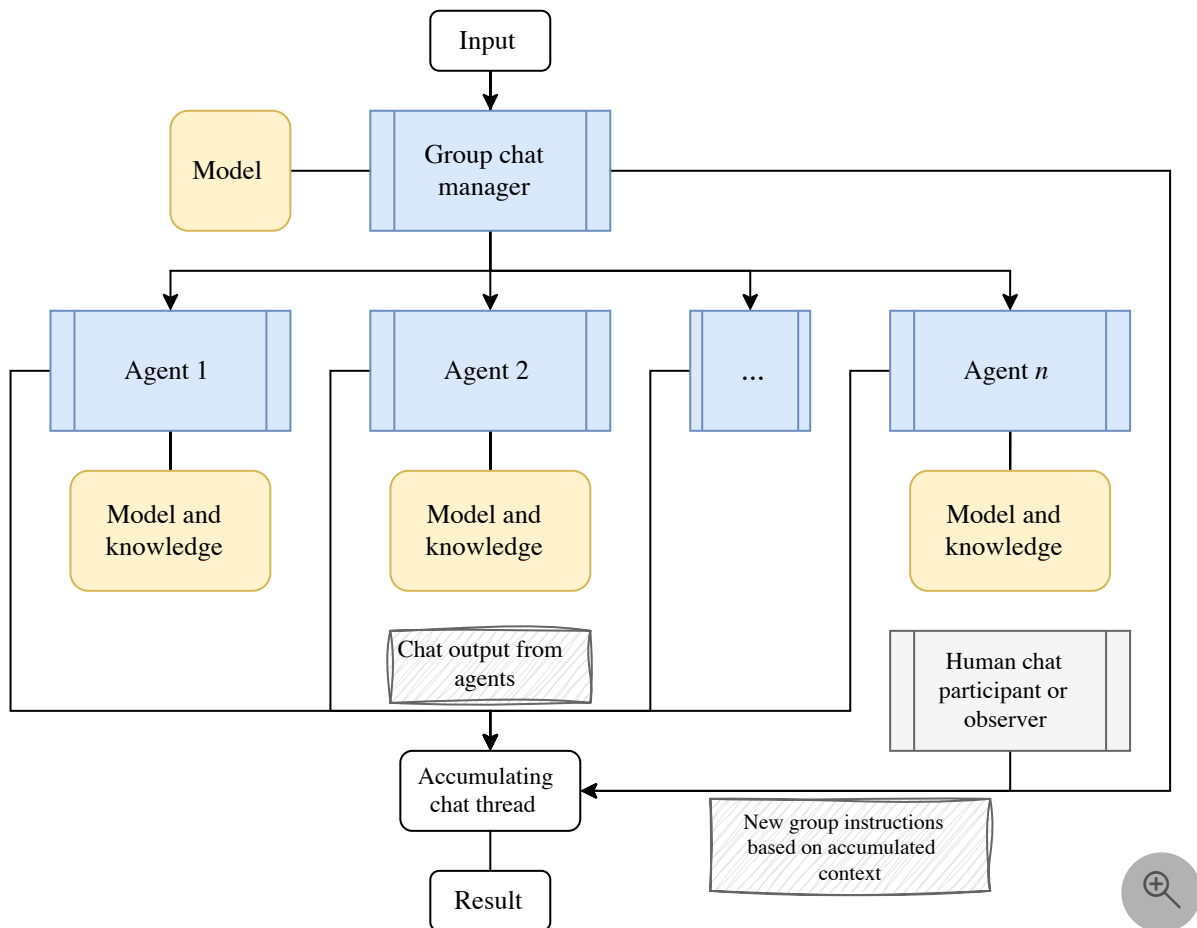
The system processes stock analysis requests by dispatching the same ticker symbol to four specialized agents that run in parallel.

- The *fundamental analysis agent* evaluates financial statements, revenue trends, and competitive positioning to assess intrinsic value.
- The *technical analysis agent* examines price patterns, volume indicators, and momentum signals to identify trading opportunities.
- The *sentiment analysis agent* processes news articles, social media mentions, and analyst reports to gauge market sentiment and investor confidence.
- The *environmental, social, and governance (ESG) agent* reviews environmental impact, social responsibility, and governance practice reports to evaluate sustainability risks and opportunities.

These independent results are then combined into a comprehensive investment recommendation, which enables portfolio managers to make informed decisions quickly.

## Group chat orchestration

The group chat orchestration pattern enables multiple agents to solve problems, make decisions, or validate work by participating in a shared conversation thread where they collaborate through discussion. A chat manager coordinates the flow by determining which agents can respond next and by managing different interaction modes, from collaborative brainstorming to structured quality gates.



This pattern addresses scenarios that are best accomplished through group discussion to reach decisions. These scenarios might include collaborative ideation, structured validation, or quality control processes. The pattern supports various interaction modes, from free-flowing brainstorming to formal review workflows that have fixed roles and approval gates.

This pattern works well for human-in-the-loop scenarios where humans can optionally take on dynamic chat manager responsibilities and guide conversations toward productive outcomes. In this orchestration pattern, agents are typically in a *read-only* mode. They don't use tools to make changes in running systems.

## When to use group chat orchestration

Consider group chat orchestration when your scenario can be solved through spontaneous or guided collaboration or iterative maker-checker loops. All of these approaches support real-time human oversight or participation. Because all agents and humans in the loop emit output into a single accumulating thread, this pattern provides transparency and auditability.

### Collaborative scenarios

- Creative brainstorming sessions where agents that have different perspectives and knowledge sources build on each other's contributions to the chat
- Decision-making processes that benefit from debate and consensus-building
- Decision-making scenarios that require iterative refinement through discussion
- Multidisciplinary problems that require cross-functional dialogue

## Validation and quality control scenarios

- Quality assurance requirements that involve structured review processes and iteration
- Compliance and regulatory validation that requires multiple expert perspectives
- Content creation workflows that require editorial review with a clear separation of concerns between creation and validation

## When to avoid group chat orchestration

Avoid this pattern in the following scenarios:

- Simple task delegation or linear pipeline processing is sufficient.
- Real-time processing requirements make discussion overhead unacceptable.
- Clear hierarchical decision-making or deterministic workflows without discussion are more appropriate.
- The chat manager has no objective way to determine whether the task is complete.

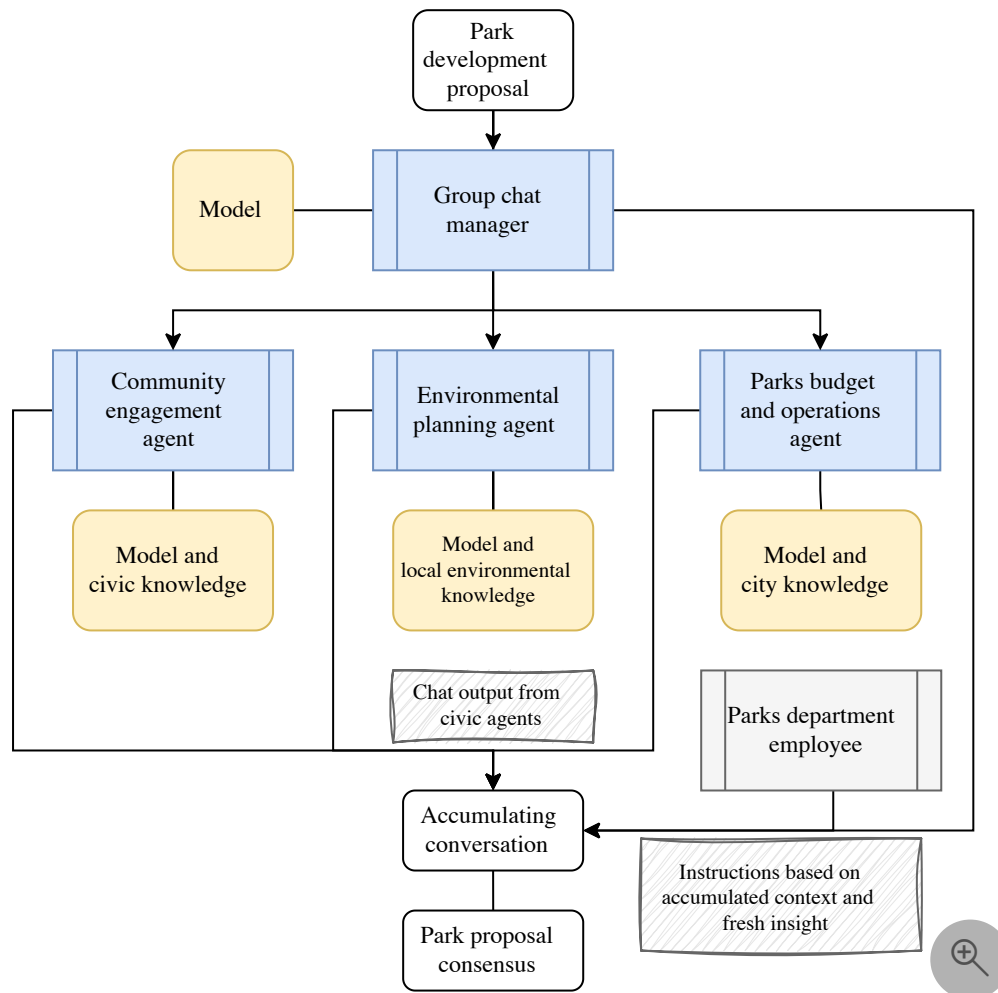
Managing conversation flow and preventing infinite loops require careful attention, especially as more agents make control more difficult to maintain. To maintain effective control, consider limiting group chat orchestration to three or fewer agents.

## Maker-checker loops

The maker-checker loop is a specific type of group chat orchestration where one agent, the *maker*, creates or proposes something. Another agent, the *checker*, provides a critique of the result. This pattern is iterative, with the checker agent pushing the conversation back to the maker agent to make updates and repeat the process. Although the group chat pattern doesn't require agents to *take turns* chatting, the maker-checker loop requires a formal turn-based sequence that the chat manager drives.

## Group chat orchestration example

A city parks and recreation department uses software that includes group chat orchestration to evaluate new park development proposals. The software reads the draft proposal, and multiple specialist agents debate different community impact perspectives and work toward consensus on the proposal. This process occurs before the proposal opens for community review to help anticipate the feedback that it might receive.



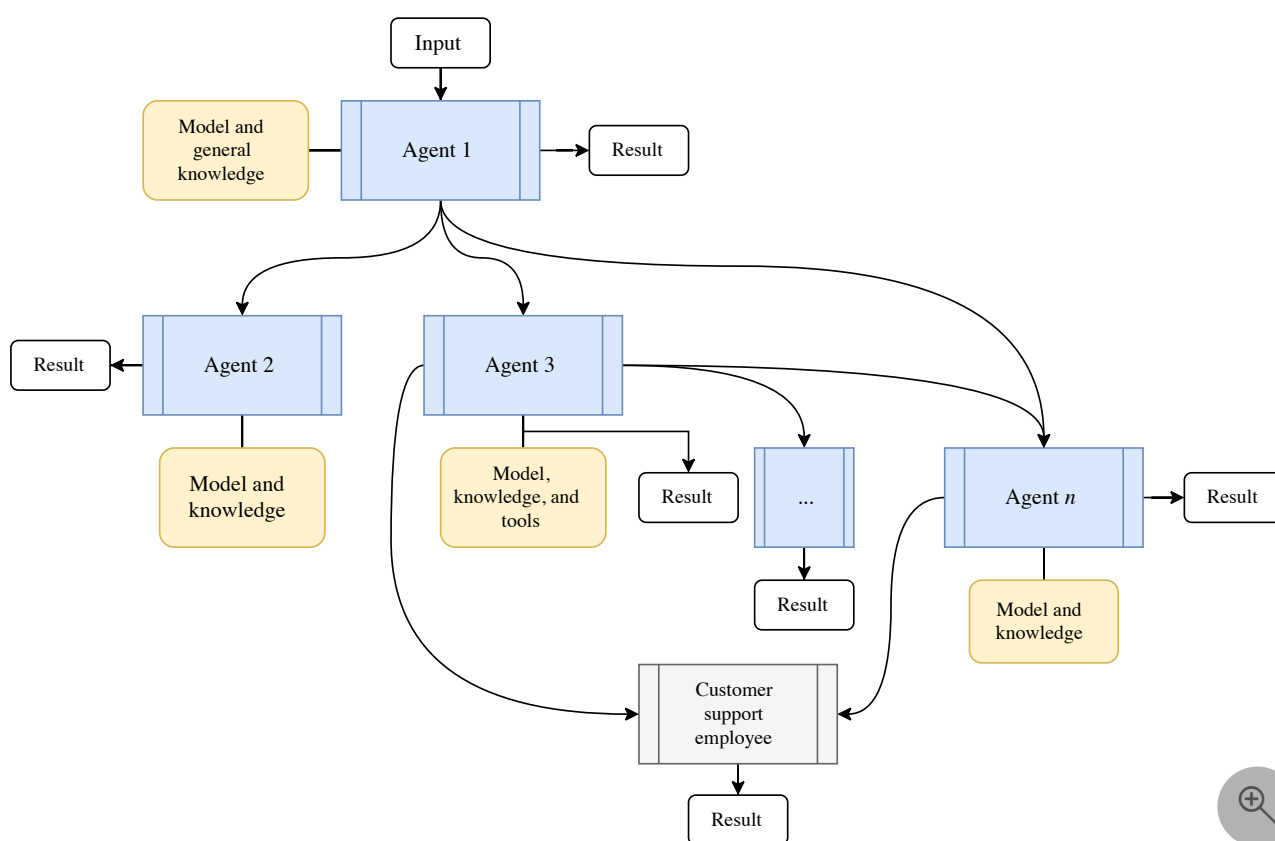
The system processes park development proposals by initiating a group consultation with specialized municipal agents that engage in the task from multiple civic perspectives.

- The *community engagement agent* evaluates accessibility requirements, anticipated resident feedback, and usage patterns to ensure equitable community access.
- The *environmental planning agent* assesses ecological impact, sustainability measures, native vegetation displacement, and compliance with environmental regulations.
- The *budget and operations agent* analyzes construction costs, ongoing maintenance expenses, staffing requirements, and long-term operational sustainability.

The chat manager facilitates structured debate where agents challenge each other's recommendations and defend their reasoning. A parks department employee participates in the chat thread to add insight and respond to agents' knowledge requests in real time. This process enables the employee to update the original proposal to address identified concerns and better prepare for community feedback.

## Handoff orchestration

The handoff orchestration pattern enables dynamic delegation of tasks between specialized agents. Each agent can assess the task at hand and decide whether to handle it directly or transfer it to a more appropriate agent based on the context and requirements.



This pattern addresses scenarios where the optimal agent for a task isn't known upfront or where the task requirements become clear only during processing. It enables intelligent routing and ensures that tasks reach the most capable agent. Agents in this pattern don't typically work in parallel. Full control transfers from one agent to another agent.

## When to use handoff orchestration

Consider the agent handoff pattern in the following scenarios:

- Tasks that require specialized knowledge or tools, but where the number of agents needed or their order can't be predetermined
- Scenarios where expertise requirements emerge during processing, resulting in dynamic task routing based on content analysis
- Multiple-domain problems that require different specialists who operate one at a time
- Logical relationships and signals that you can predetermine to indicate when one agent reaches its capability limit and which agent should handle the task next

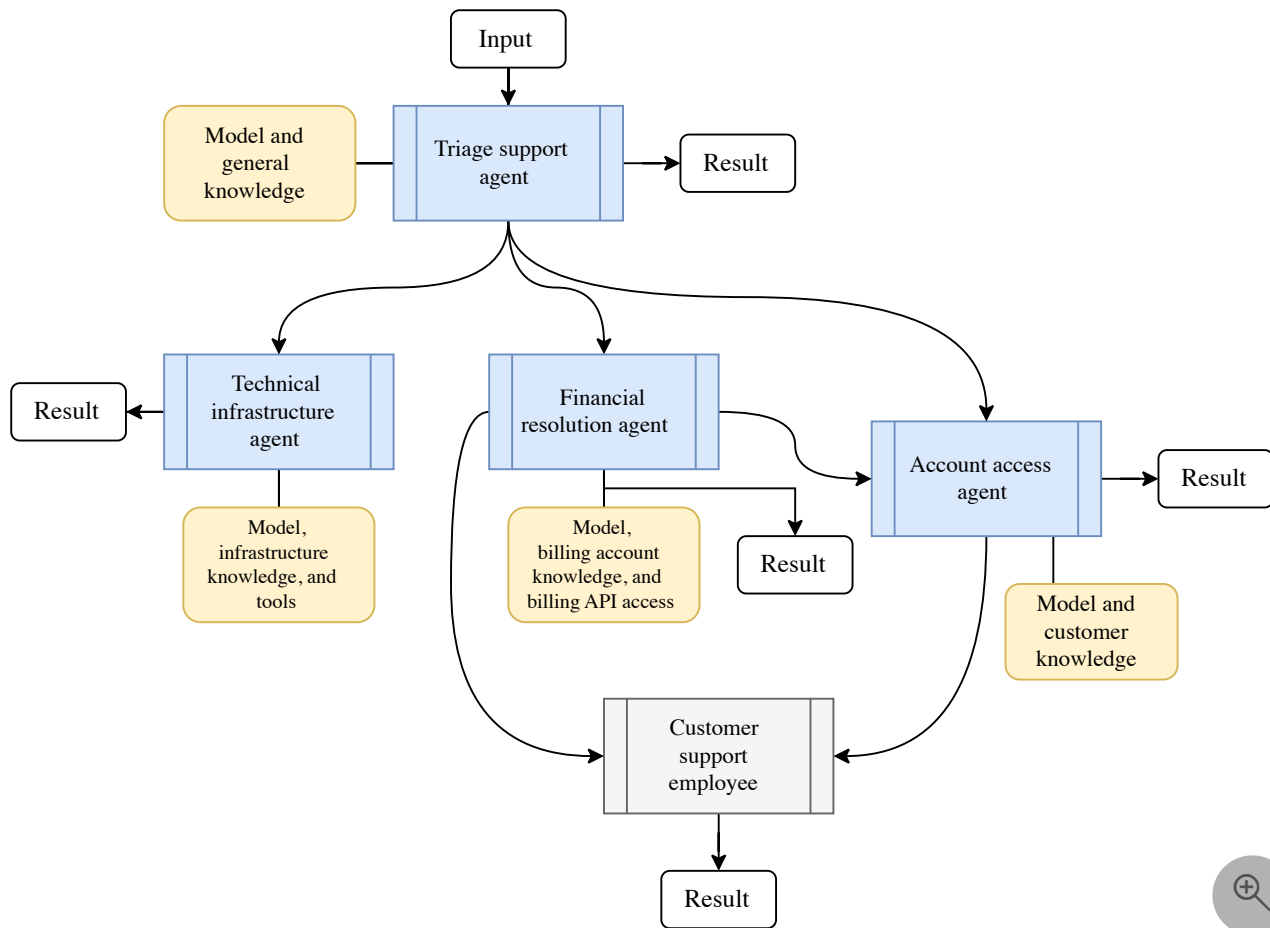
## When to avoid handoff orchestration

Avoid this pattern in the following scenarios:

- The appropriate agents and their order are always known upfront.
- Task routing is simple and deterministically rule-based, not based on dynamic context window or dynamic interpretation.
- Suboptimal routing decisions might lead to a poor or frustrating user experience.
- Multiple operations should run concurrently to address the task.
- Avoiding an infinite handoff loop or avoiding excessive bouncing between agents is challenging.

## Agent handoff pattern example

A telecommunications customer relationship management (CRM) solution uses handoff agents in its customer support web portal. An initial agent begins helping customers but discovers that it needs specialized expertise during the conversation. The initial agent passes the task to the most appropriate agent to address the customer's concern. Only one agent at a time operates on the original input, and the handoff chain results in a single result.



In this system, the *triage support agent* interprets the request and tries to handle common problems directly. When it reaches its limits, it hands network problems to a *technical infrastructure agent*, billing disputes to a *financial resolution agent*, and so on. Further handoffs occur within those agents when the current agent recognizes its own capability limits and knows another agent can better support the scenario.

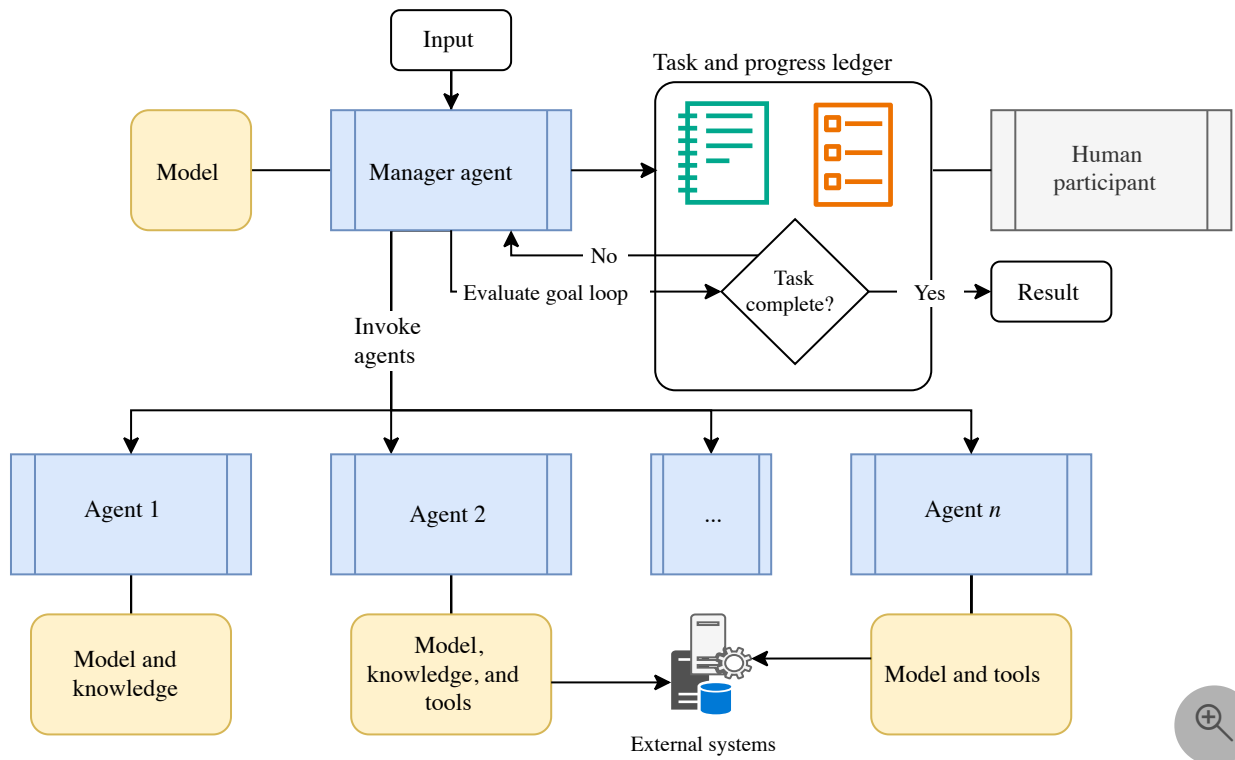
Each agent is capable of completing the conversation if it determines that customer success has been achieved or that no other agent can further benefit the customer. Some agents are also designed to hand off the user experience to a human support agent when the problem is important to solve but no AI agent currently has the capabilities to address it.

One example of a handoff instance is highlighted in the diagram. It begins with the triage agent that hands off the task to the technical infrastructure agent. The technical infrastructure agent then decides to hand off the task to the financial resolution agent, which ultimately redirects the task to customer support.

## Magentic orchestration

The magentic orchestration pattern is designed for open-ended and complex problems that don't have a predetermined plan of approach. Agents in this pattern typically have tools that allow them to make direct changes in external systems. The focus is as much on building and

documenting the approach to solve the problem as it is on implementing that approach. The task list is dynamically built and refined as part of the workflow through collaboration between specialized agents and a magentic manager agent. As the context evolves, the magentic manager agent builds a task ledger to develop the approach plan with goals and subgoals, which is eventually finalized, followed, and tracked to complete the desired outcome.



The manager agent communicates directly with specialized agents to gather information as it builds and refines the task ledger. It iterates, backtracks, and delegates as many times as needed to build a complete plan that it can successfully carry out. The manager agent frequently evaluates whether the original request is fully satisfied or stalled. It updates the ledger to adjust the plan.

In some ways, this orchestration pattern is an extension of the [group chat](#) pattern. The magentic orchestration pattern focuses on an agent that builds a plan of approach, while other agents use tools to make changes in external systems instead of only using their knowledge stores to reach an outcome.

## When to use magentic orchestration

Consider the magentic pattern in the following scenarios:

- A complex or open-ended use case that has no predetermined solution path.
- A requirement to consider input and feedback from multiple specialized agents to develop a valid solution path.

- A requirement for the AI system to generate a fully developed plan of approach that a human can review before or after implementation.
- Agents equipped with tools that interact with external systems, consume external resources, or can induce changes in running systems. A documented plan that shows how those agents are sequenced can be presented to a user before allowing the agents to follow the tasks.

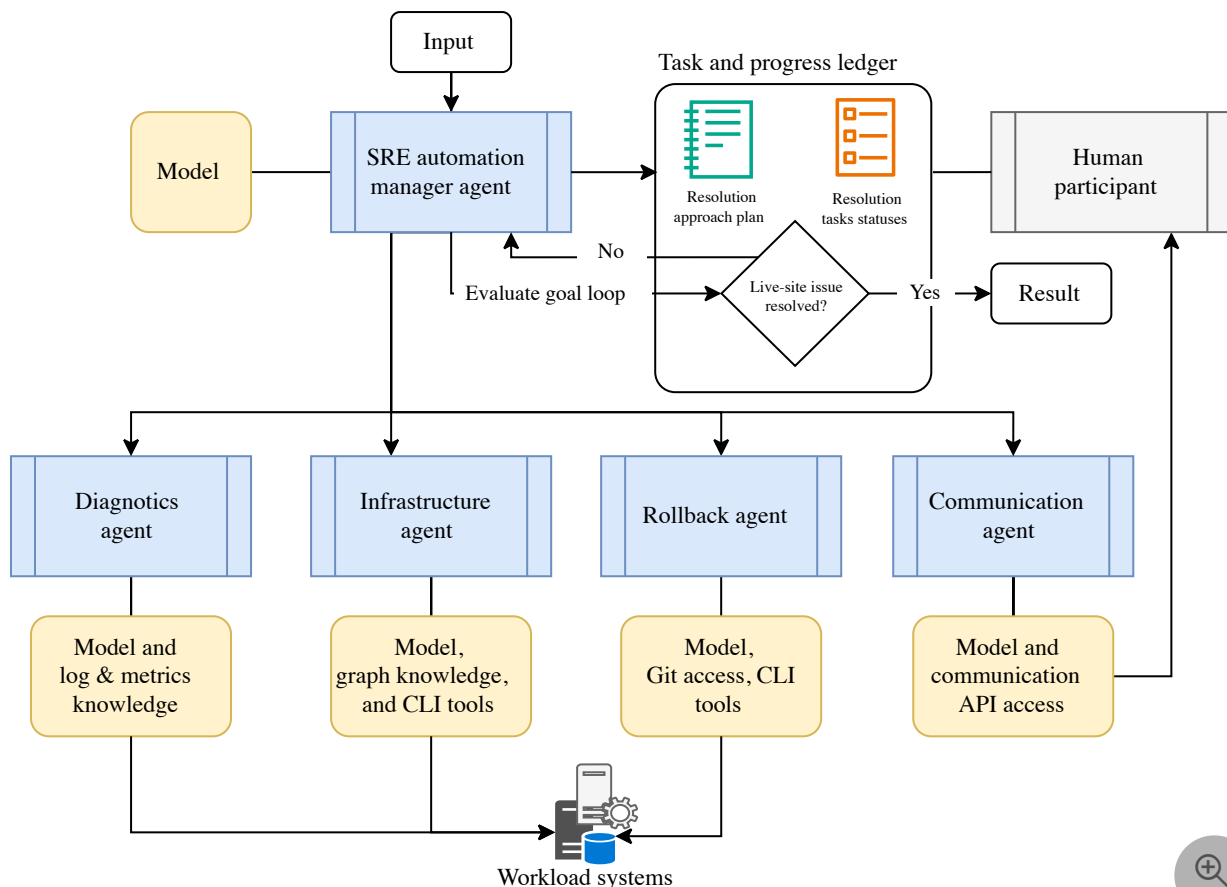
## When to avoid magentic orchestration

Avoid this pattern in the following scenarios:

- The solution path is developed or should be approached in a deterministic way.
- There's no requirement to produce a ledger.
- The task has low complexity and a simpler pattern can solve it.
- The work is time-sensitive, as the pattern focuses on building and debating viable plans, not optimizing for end results.
- You anticipate frequent stalls or infinite loops that don't have a clear path to resolution.

## Magentic orchestration example

A site reliability engineering (SRE) team built automation that uses magentic orchestration to handle low-risk incident response scenarios. When a service outage occurs within the scope of the automation, the system must dynamically create and implement a remediation plan. It does this without knowing the specific steps needed upfront.



When the automation detects a qualifying incident, the *magentic manager agent* begins by creating an initial task ledger with high-level goals such as restoring service availability and identifying the root cause. The manager agent then consults with specialized agents to gather information and refine the remediation plan.

1. The *diagnostics agent* analyzes system logs, performance metrics, and error patterns to identify potential causes. It reports findings back to the manager agent.
2. Based on diagnostic results, the manager agent updates the task ledger with specific investigation steps and consults the *infrastructure agent* to understand current system state and available recovery options.
3. The *communication agent* provides stakeholder notification capabilities, and the manager agent incorporates communication checkpoints and approval gates into the evolving plan according to the SRE team's escalation procedures.
4. As the scenario becomes clearer, the manager agent might add the *rollback agent* to the plan if deployment reversion is needed, or escalate to human SRE engineers if the incident exceeds the automation's scope.

Throughout this process, the manager agent continuously refines the task ledger based on new information. It adds, removes, or reorders tasks as the incident evolves. For example, if the

diagnostics agent discovers a database connection problem, the manager agent might switch the entire plan from a deployment rollback strategy to a plan that focuses on restoring database connectivity.

The manager agent watches for excessive stalls in restoring service and guards against infinite remediation loops. It maintains a complete audit trail of the evolving plan and the implementation steps, which provides transparency for post-incident review. This transparency ensures that the SRE team can improve both the workload and the automation based on lessons learned.

## Implementation considerations

When you implement any of these agent design patterns, several considerations must be addressed. Reviewing these considerations helps you avoid common pitfalls and ensures that your agent orchestration is robust, secure, and maintainable.

### Single agent, multitool

You can address some problems with a single agent if you give it sufficient access to tools and knowledge sources. As the number of knowledge sources and tools increases, it becomes difficult to provide a predictable agent experience. If a single agent can reliably solve your scenario, consider adopting that approach. Decision-making and flow-control overhead often exceed the benefits of breaking the task into multiple agents. However, security boundaries, network line of sight, and other factors can still render a single-agent approach infeasible.

### Deterministic routing

Some patterns require you to route flow between agents deterministically. Others rely on agents to choose their own routes. If your agents are defined in a no-code or low-code environment, you might not control those behaviors. If you define your agents in code by using SDKs like [Microsoft Agent Framework](#) or Semantic Kernel, you have more control.

### Context window

AI agents often have limited context windows. This constraint can affect their ability to process complex tasks. When you implement these patterns, decide what context the next agent requires to be effective. In some scenarios, you need the full, raw context gathered so far. In other scenarios, a summarized or truncated version is more appropriate. If your agent can work without accumulated context and only requires a new instruction set, take that approach instead of providing context that doesn't help accomplish the agent's task.

## Reliability

These patterns require properly functioning agents and reliable transitions between them. They often result in classical distributed systems problems such as node failures, network partitions, message loss, and cascading errors. Mitigation strategies should be in place to address these challenges. Agents and their orchestrators should do the following steps.

- Implement timeout and retry mechanisms.
- Include a graceful degradation implementation to handle one or more agents within a pattern faulting.
- Surface errors instead of hiding them, so downstream agents and orchestrator logic can respond appropriately.
- Consider circuit breaker patterns for agent dependencies.
- Design agents to be as isolated as is practical from each other, with single points of failure not shared between agents. For example:
  - Ensure compute isolation between agents.
  - Evaluate how using a single models as a service (MaaS) model or a shared knowledge store can result in rate limiting when agents run concurrently.
- Use checkpoint features available in your SDK to help recover from an interrupted orchestration, such as from a fault or a new code deployment.

## Security

Implementing proper security mechanisms in these design patterns minimizes the risk of exposing your AI system to attacks or data leakage. Securing communication between agents and limiting each agent's access to sensitive data are key security design strategies. Consider the following security measures:

- Implement authentication and use secure networking between agents.
- Consider data privacy implications of agent communications.
- Design audit trails to meet compliance requirements.
- Design agents and their orchestrators to follow the principle of least privilege.
- Consider how to handle the user's identity across agents. Agents must have broad access to knowledge stores to handle requests from all users, but they must not return data

that's inaccessible to the user. Security trimming must be implemented in every agent in the pattern.

## Observability and testing

Distributing your AI system across multiple agents requires monitoring and testing each agent individually, as well as the system as a whole, to ensure proper functionality. When you design your observability and testing strategies, consider the following recommendations:

- Instrument all agent operations and handoffs. Troubleshooting distributed systems is a computer science challenge, and orchestrated AI agents are no exception.
- Track performance and resource usage metrics for each agent so that you can establish a baseline, find bottlenecks, and optimize.
- Design testable interfaces for individual agents.
- Implement integration tests for multi-agent workflows.

## Common pitfalls and anti-patterns

Avoid these common mistakes when you implement agent orchestration patterns:

- Creating unnecessary coordination complexity by using a complex pattern when simple sequential or concurrent orchestration would suffice.
- Adding agents that don't provide meaningful specialization.
- Overlooking latency impacts of multiple-hop communication.
- Sharing mutable state between concurrent agents, which can result in transactionally inconsistent data because of assuming synchronous updates across agent boundaries.
- Using deterministic patterns for workflows that are inherently nondeterministic.
- Using nondeterministic patterns for workflows that are inherently deterministic.
- Ignoring resource constraints when you choose concurrent orchestration.
- Consuming excessive model resources because context windows grow as agents accumulate more information and consult their model to make progress on their task.

## Combining orchestration patterns

Applications sometimes require you to combine multiple orchestration patterns to address their requirements. For example, you might use sequential orchestration for the initial data processing stages and then switch to concurrent orchestration for parallelizable analysis tasks. Don't try to make one workflow fit into a single pattern when different stages of your workload have different characteristics and can benefit from each stage using a different pattern.

## Relationship to cloud design patterns

AI agent orchestration patterns extend and complement traditional [cloud design patterns](#) by addressing the unique challenges of coordinating intelligent, autonomous components. Cloud design patterns focus on structural and behavioral concerns in distributed systems, but AI agent orchestration patterns specifically address the coordination of components with reasoning capabilities, learning behaviors, and nondeterministic outputs.

## SDK-based implementations

Many of these patterns rely on a code-based implementation to address the orchestration logic. SDKs that support an agent framework often provides support for many of the agent orchestration patterns.

## Microsoft Agent Framework

The Microsoft Agent Framework SDK has implementation guidance for [Agent Framework orchestration](#).

- [Sequential orchestration using the Agent Framework](#)
- [Concurrent orchestration using the Agent Framework](#)
- [Handoff orchestration using the Agent Framework](#)
- [Magentic orchestration using the Agent Framework](#)

For hands-on implementation, explore [Agent Framework declarative workflow samples](#) [↗](#) on GitHub that demonstrate some of these patterns in practice.

## Semantic Kernel

The Semantic Kernel SDK has implementation guidance for its [agent framework](#).

- [Sequential orchestration using Semantic Kernel](#)
- [Concurrent orchestration using Semantic Kernel](#)
- [Group Chat orchestration using Semantic Kernel](#)
- [Handoff orchestration using Semantic Kernel](#)

- [Magentic orchestration using Semantic Kernel](#)

For hands-on implementation, explore [Semantic Kernel multi-agent orchestration samples](#) on GitHub that demonstrate these patterns in practice.

You can also find many of these patterns in [AutoGen](#), such as [Magentic-One](#).

## Implementations in Azure AI Foundry Agent Service

You can also use the [Azure AI Foundry Agent Service](#) to chain agents together in relatively simple workflows by using its [connected agents](#) functionality. The workflows that you implement by using this service are primarily nondeterministic, which limits which patterns can be fully implemented in this no-code environment.

## Contributors

*Microsoft maintains this article. The following contributors wrote this article.*

Principal authors:

- [Chad Kittel](#) | Principal Software Engineer - Azure Patterns & Practices
- [Clayton Siemens](#) | Principal Content Developer - Azure Patterns & Practices

Other contributors:

- [Hemavathy Alaganandam](#) | Principal Software Engineer
- [James Lee](#) | Data Scientist 2
- [Ritesh Modi](#) | Principal Software Engineer
- [Mahdi Setayesh](#) | Principal Software Engineer
- [Mark Taylor](#) | Principal Software Engineer
- [Yaniv Vaknin](#) | Senior Technical Specialist

*To see nonpublic LinkedIn profiles, sign in to LinkedIn.*

## Next step

[Implement agent orchestration with Semantic Kernel](#)

Last updated on 07/19/2025